

Lab 7 — CI/CD Integration Testing

Track B (BMD-45 vehicle detection) · Week 7 · DS5619 Machine Learning Systems Operations

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

This overwrites `data/fixtures/` with your own synthetic CCTV images — same file names, same two camera profiles, seeded deterministically from your student ID. It doesn't change what you're graded on this week (that's `ci.yml` and `integration_test.sh`, both plumbing, not data-dependent), but your `integration-test` CI job ends up curling YOUR image, not a shared one.

Record your `--student-id` value in `NOTES.md`.

Learning objective

Write a CI pipeline that gates merges on three things actually passing: lint, unit tests, and an integration test against the real built container — not just "the code runs on my machine." This is the shift-left / CI content from this week's lecture: catch a broken Dockerfile or a bad `/detect` regression in a pull request, not after it's already deployed.

Files

- `src/app.py`, `Dockerfile` — Week 6's already-solved app + container. Don't edit.
- `.github/workflows/ci.yml` — fill in the `???` blanks.
- `scripts/integration_test.sh` — fill in the `???` blanks.
- `CI_VERIFICATION.md` — paste your green Actions run URL here.

Why the app is already done

`src/app.py` and `Dockerfile` are Week 6's already-solved API and container — copied in as-is so this week's 110 minutes goes entirely into the CI pipeline, not re-solving last week's lab. Don't edit them.

Your task

Part 1 — `.github/workflows/ci.yml` (several `???`/TODO blanks)

Three jobs: `lint` (flake8), `unit-test` (pytest), `integration-test` (builds + runs the container, calls `scripts/integration_test.sh`). `integration-test` must declare `needs: [lint, unit-test]` — it shouldn't spend CI minutes building a container for code that doesn't even lint.

Part 2 — `scripts/integration_test.sh` (three `???` blanks)

The actual container test: `docker build`, `docker run` (detached, port mapped), poll `/health` until it responds, `curl /detect` with a sample fixture image and check the response contains "`detections`", then tear the container down. Test this locally before wiring it into CI:

```
chmod +x scripts/integration_test.sh
./scripts/integration_test.sh
```

Part 3 — Push and watch it run

Push to GitHub, open the Actions tab, confirm all three jobs go green (fix anything that doesn't). Fill in `CI_VERIFICATION.md` with the run URL.

Self-check

```
pytest tests/ -q
```

`tests/test_app.py` checks the (already-complete) API. `tests/test_ci_config.py` checks your `ci.yml/integration_test.sh` structurally — it's a self-check, not a substitute for actually pushing and watching Actions run.

Deliverables (what you commit)

- `.github/workflows/ci.yml`, completed.
- `scripts/integration_test.sh`, completed.
- `CI_VERIFICATION.md`, filled in with a real run URL (a run that actually succeeded — a red run doesn't count).
- A short `NOTES.md`: the `--student-id` value you used (required — see above), and why does `integration-test` need `needs: [lint, unit-test]` instead of just running in parallel with them — what's the actual cost being avoided?

Grading checklist

- ☐ `data/fixtures/` matches what `generate_for_student.py --student-id <NOTES.md value>` actually produces.
- ☐ `ci.yml` defines exactly the three required jobs with the correct `needs:` dependency (checked structurally by `tests/test_ci_config.py`, not the full grader).
- ☐ `scripts/integration_test.sh` is valid bash, references `docker build`, `docker run`, and `curl`, and actually works when run locally.
- ☐ `CI_VERIFICATION.md` links to a genuinely green run of all three jobs on GitHub (the grader will open the link).
- ☐ `NOTES.md` reasons about the actual cost of skipping the `needs:` gate.
- ☐ Meaningful commit history and a working README.

Submission

```
git add -A  
git commit -m "Week 7: CI/CD integration testing"  
git tag week07-submit  
git push origin main --tags
```

Source: this lab operationalizes the CI/CD, Integration Testing, and Shift-Left content from the Week 7 lecture deck.